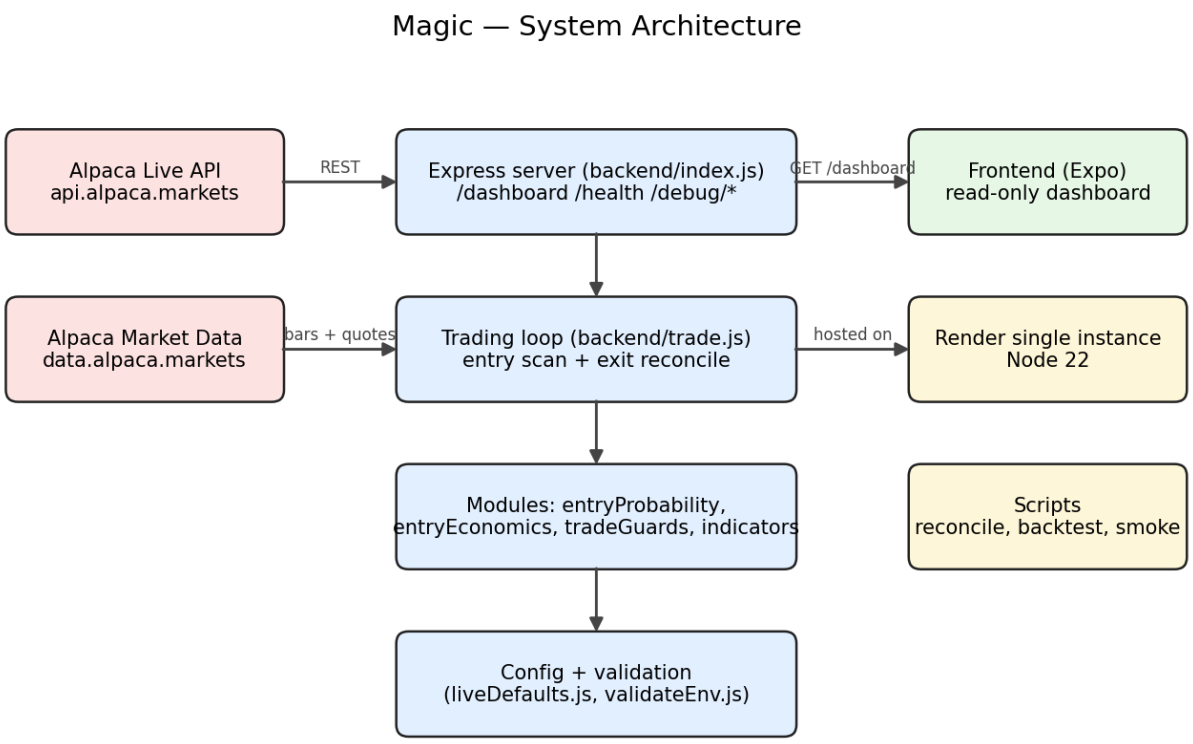
Magic

Alpaca Crypto Trading Bot — Technical Specification



Generated 2026-05-13 - source of truth: README.md at repo root

WARNING: This is a live trading system. Real money is at risk every time it runs. Treat this document as a specification of intended behaviour; the canonical reference is `README.md` in the repository root.

1. Overview

Magic is an automated cryptocurrency trading bot built on Alpaca's **live** trading API. Every few seconds it scans a configured set of USD-quoted crypto pairs, opens a small long position when recent price action looks favourable, and immediately parks a take-profit limit on fill. It runs unattended on a single Render instance.

1.1 Goals

- Detect tiny upward drifts in liquid crypto pairs (sub-minute OLS regression).
- Capture a small **net** profit per trade after fees (default **8 bps** floor, per-trade target sized from the signal's projected move, clamped to [8, 50] bps).
- **Never realise a loss by default**. The GTC sell limit is walked down toward break-even-after-fees over `BREAKEVEN_TIMEOUT_MS` (4h) and pinned there - worst-case realised P&L per trade is $0 net.
- Concurrency is bounded by available cash, not a fixed slot count (`PORTFOLIO_SIZING_PCT` of equity per trade).
- Single Render instance; per-process in-memory rate limiting; 24/7 operation.

1.2 Non-goals (intentional)

- No leverage, no averaging down, no pyramiding.
- No trailing stop. Even when `STOP_LOSS_ENABLED=true`, the stop is static at fill time.
- No realised loss while `STOP_LOSS_ENABLED=false` (the default). The staircase exit floors realised P&L at $0 net.
- No Kelly sizing, kill-switch file watcher, TWAP execution, or cross-symbol correlation guard.
- Older doc fragments mention these features; treat any env var not listed in this spec or in README.md as not wired until confirmed by `grep`.

2. System architecture

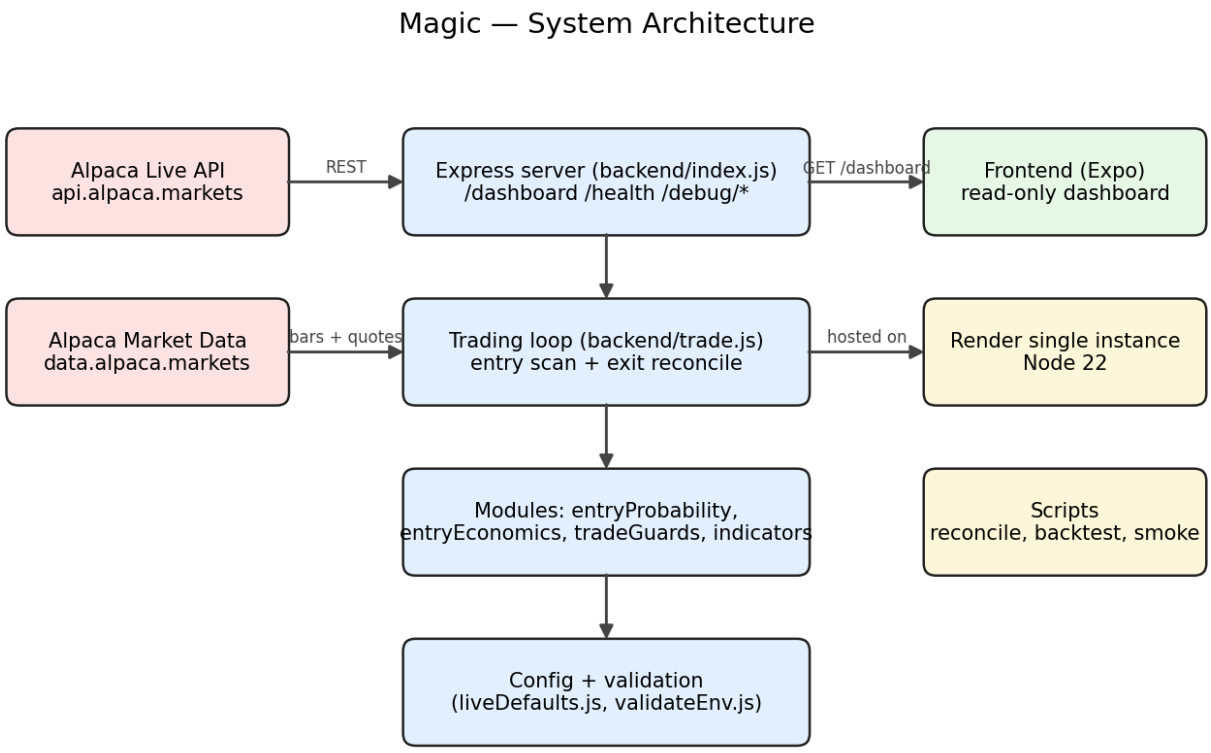


Figure 1. Backend talks to Alpaca's live REST API for orders and to the market-data API for bars and quotes. The Expo frontend polls the backend's read-only dashboard endpoint. Operational scripts live alongside the backend.

2.1 Repo layout

Repo layout

backend/	Node 22 Express engine — REST + trading loop
backend/trade.js	Main scan/predict/gate/buy/exit loop (~2.7k LOC)
backend/index.js	Express server, routes, dashboard meta
backend/modules/	Math + helpers (entryProbability, tradeGuards, ...)
backend/config/	Live defaults, runtime config, env validation
backend/scripts/	reconcile, backtest, runtime-env check, smoke
Frontend/	Expo read-only dashboard polling /dashboard
shared/	symbol normalization + quote utils shared by both
scripts/	Repo tooling (git-hook installer)
.git-hooks/	Pre-commit secret scanner
.github/workflows/	CI: lint + tests + env check, frontend smoke
docs/	This spec doc + generator + images

Figure 2. Top-level directories and their roles.

3. Strategy

The full strategy is six steps. Everything else in the codebase is plumbing, telemetry, and safety rails around these.

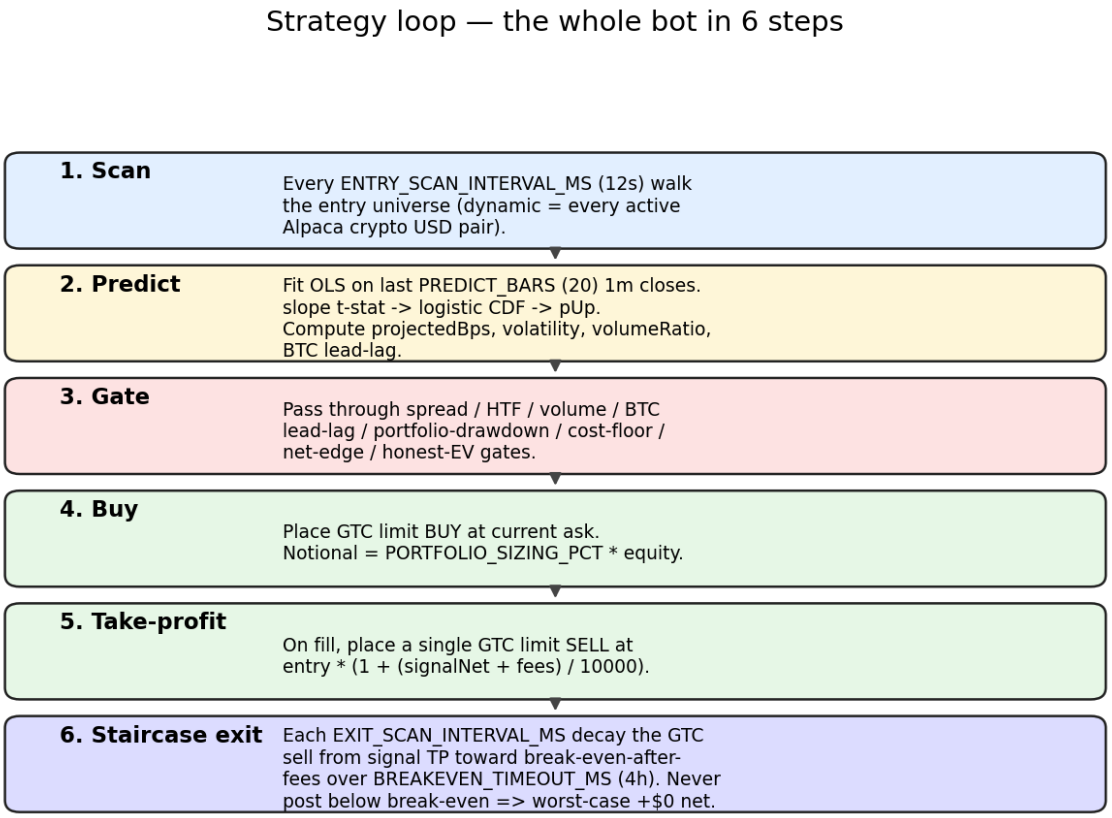


Figure 3. End-to-end strategy loop.

3.1 Entry math

Entry signal lives in `backend/modules/entryProbability.js`: fit OLS on the last `PREDICT_BARS` (20) one-minute closes, convert the slope's t-statistic to an upward probability via the logistic CDF.

Forward fill probability lives in `backend/modules/entryEconomics.js`: closed-form GBM barrier-hitting probability that the bid reaches the take-profit price within `BARRIER_HORIZON_BARS` (defaults to `BREAKEVEN_TIMEOUT_MS` in minutes). Drift mu comes from the OLS slope, sigma from recent realised 1m volatility. This replaced the older `logistic_cdf(slopeTStat)` proxy in 2026; set `CORRECTED_FILL_PROB_ENABLED=false` to roll back.

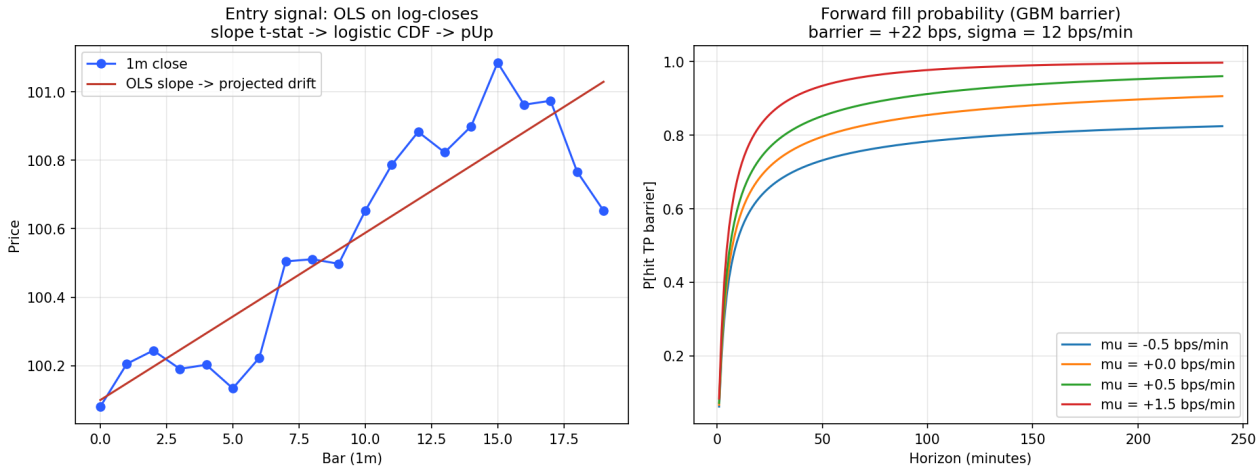


Figure 4. Left: per-symbol OLS regression on recent 1m closes. Right: forward barrier-hitting probability as a function of horizon and drift, at sigma = 12 bps/min, barrier = +22 bps.

4. Entry gates

A candidate must clear every gate in this funnel before a GTC limit BUY is placed. Most are wired in `backend/modules/tradeGuards.js`; the macro portfolio-drawdown gate lives in `backend/trade.js` alongside the scan loop.

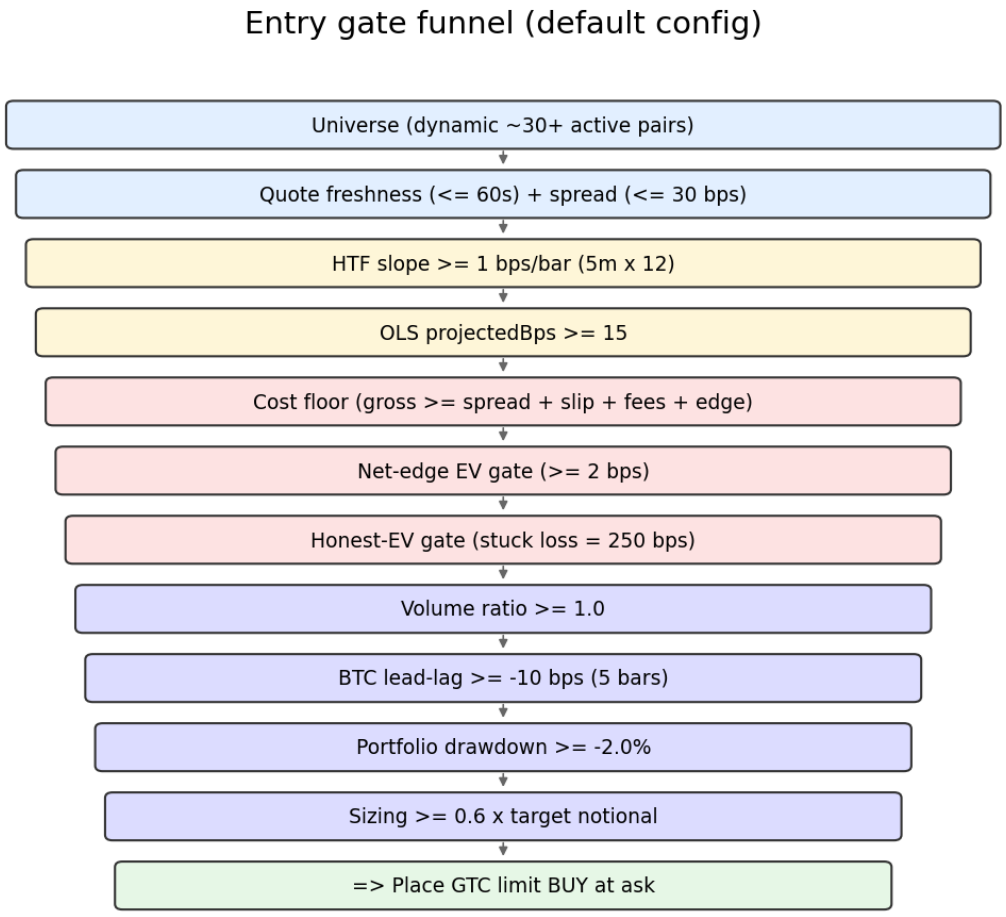


Figure 5. Entry gate funnel under default configuration. Width is illustrative (not measured) - it conveys progressively tighter filtering, not surviving fractions.

4.1 Gate reference

Gate	Default	Skip reason
Quote freshness	<= 60 s	quote_stale
Spread	<= 30 bps	spread_too_wide
HTF slope (5m x 12)	>= 1 bps/bar	htf_slope_too_low
Projected forward move	>= 15 bps	projected_below_min
Cost floor	gross >= friction sum	gross_target_below_friction_floor
Net-edge EV	>= 2 bps	net_edge_below_min
Honest-EV (stuck=250 bps)	>= MIN_NET_EDGE_BPS	honest_ev_below_min
Volume ratio	>= 1.0	volume_below_min

Gate	Default	Skip reason
BTC lead-lag (5 bars)	>= -10 bps	btc_leading_drop
Portfolio drawdown %	>= -2.0%	portfolio_drawdown_below_min
Sizing fraction	>= 0.6 of target	sizing_below_floor
Volatility	<= 100 bps	volatility_too_high

5. Exit behaviour

On buy fill the engine places **one** GTC limit SELL at `entry x (1 + (signalNet + fees) / 10000)`, where `signalNet = clamp(SIGNAL_TARGET_FRACTION x projectedBps - fees, TARGET_NET_PROFIT_BPS, SIGNAL_TARGET_MAX_NET_BPS)`. Confident signals get bigger targets; marginal signals fall back to the 8 bps floor.

5.1 Staircase exit (no realised losses)

Each `EXIT_SCAN_INTERVAL_MS` reconcile cycle, the engine computes a desired GTC sell limit that decays linearly from the signal-derived TP at fill time toward break-even-after-fees (`entry x (1 + FEE_BPS_ROUND_TRIP/10000)`) over `BREAKEVEN_TIMEOUT_MS` (4h by default). When the desired price drops at least `STAIRCASE_REPOST_TOLERANCE_BPS` (3) below the resting limit, the engine cancels and reposts at the lower price. The floor is the break-even-after-fees price - the bot **never** reposts below it, so every fill yields >= $0 net.

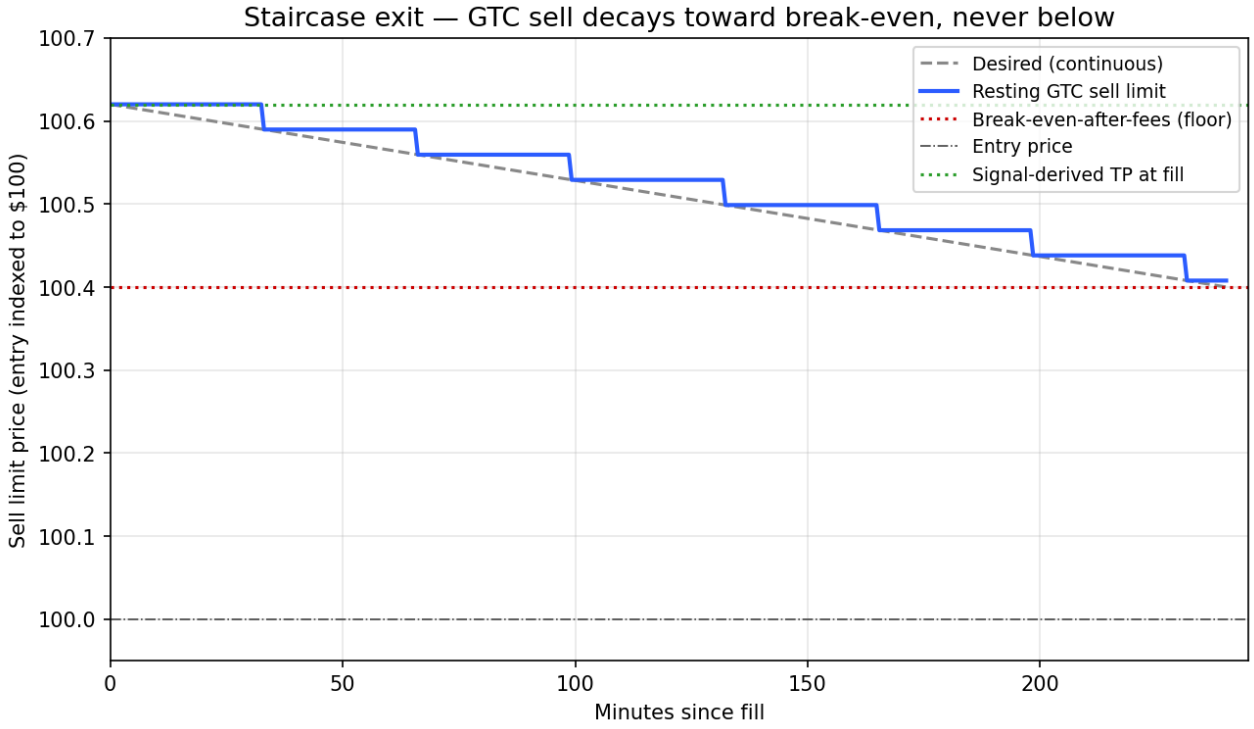


Figure 6. The resting GTC sell limit decays toward break-even-after-fees over 4 hours. Steps reflect the ~3-bps repost tolerance that prevents cancel/repost churn on tiny age increments. The break-even floor is hard.

5.2 Stop-loss (opt-in)

`STOP_LOSS_ENABLED=false` is the default. When flipped on, the exit manager monitors the live bid and force-exits with a market IOC sell if the stop is breached - i.e. the bot will realise a loss. With `VOL_SCALED_STOP_ENABLED=true`, per-trade stop distance is sized from entry-time volatility: `stopBps ~ STOP_LOSS_VOL_K * sigma * sqrt(STOP_LOSS_HORIZON_BARS)`, clamped to `[STOP_LOSS_BPS_FLOOR, STOP_LOSS_BPS]`.

Restart resilience: the staircase age anchor uses the older of (broker GTC sell `created_at`, in-memory `positionFirstSeenAt`) - positions opened well before a deploy resume their decay instead of resetting to t=0.

6. Top-detection features

Four features are computed every scan and dropped into the `entry_submitted` log and dashboard `forensics` payload.

Field	Wired as gate?	Meaning
volumeRatio	Yes — MIN_VOLUME_RATIO_TO_ENTER	mean(last-25%-window 1m volume) / mean(all PREDICT_BARS 1m volume). >1 = volume rising in the recent window; <1 = fading. Tops typically print on declining volume.
volumeWeightedSlopeBps	No — forensics only	Same OLS slope but each bar weighted by its volume. Disagreement with the unweighted slope means the trend is being pushed by low-volume noise.
btcLeadLag.{recentReturnBps, slopeBpsPerBar, ageMs}	Yes — MAX_BTC_LEAD_LAG_DROP_BPS	BTC's last-5-bar return attached to every non-BTC entry. Alts lag BTC by 30–90s; a fresh BTC drop is a leading indicator that alt momentum is about to reverse.
bookImbalance	No — forensics only when ORDERBOOK_IMBALANCE_FEATURE_ENABLED	Top-N orderbook notional imbalance in [-1, +1]. Disabled by default because it costs an extra /latest/orderbooks fetch per symbol against the 200/min budget.

6.1 Automatic backtests

The bot runs the backtester automatically ~60 s after every server start, against the last `BACKTEST_AUTORUN_DAYS` (30) of bars for the configured universe. The result is parked in memory and surfaced under `meta.backtest` on `/dashboard`.

After the primary run completes, two alt runs fire, each isolating one top-detection gate so per-gate expectancy impact is attributable: **alt** = looser BTC lead-lag gate ON, volume gate OFF; **alt2** = tighter volume gate ON, BTC gate OFF. Compare `overall.avgNetBpsPerEntry` between primary, alt, and alt2 to see which gate (if any) improves expectancy on real history before flipping it on live.

On-demand parameter sweeps via the same path:

```
GET /debug/backtest -> cached result if any
GET /debug/backtest?refresh=true -> re-run with default params
GET /debug/backtest?days=60&signalTargetFraction=1.0 -> re-run with overrides (waits)
GET /debug/backtest?wait=false&minProjectedBps=25 -> kick off in background
```

7. Expectancy

Two analysis scripts answer "is this strategy actually profitable?": `npm run reconcile` compares predicted vs realised on live forensics data, and `node scripts/simulate_strategy.js` runs a closed-form Monte Carlo across drift/vol regimes.

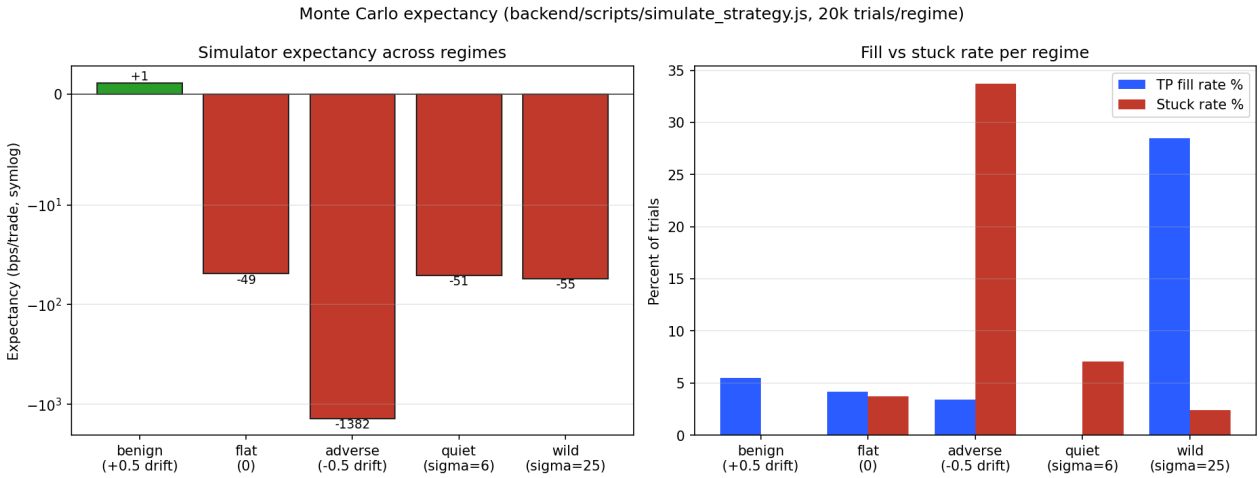


Figure 7. Simulator output (20 000 trials/regime, default fees/spread, 10-min break-even timeout). Expectancy is **strongly negative under flat or adverse drift** because stuck positions accumulate negative MTM that the engine never crystallises.

Regime	Drift (bps/min)	TP fill rate	Stuck rate	Expectancy (bps/trade)
benign	+0.5	5.5%	0.0%	+1.00
flat	0	4.2%	3.7%	-49
adverse	-0.5	3.4%	33.7%	-1382
quiet	0 (sigma=6)	0.0%	7.1%	-51
wild	0 (sigma=25)	28.5%	2.4%	-55

7.1 Three responses if production expectancy stays negative

- Widen `TARGET_NET_PROFIT_BPS` materially (50-80 bps) so winners pay for the stuck tail. The simulator shows this alone is insufficient - fill rates collapse roughly proportionally.
- Keep `HONEST_EV_GATE_ENABLED=true` and tune `STUCK_LOSS_ASSUMED_BPS` to match observed adverse-regime MTM, accepting that this starves entries in any regime that isn't trending up.
- Tighten `STOP_LOSS_BPS` and/or shorten `BREAKEVEN_TIMEOUT_MS` for faster loss realization and capital recycling in adverse regimes.

8. Configuration reference

Defaults below match `README.md` (which is the source of truth) and `backend/config/liveDefaults.js`. If you see env vars referenced in older doc fragments that aren't listed here, treat them as **not wired** until confirmed by `grep` in `backend/`.

8.1 Required for live trading

Variable	Purpose
APCA_API_KEY_ID	Alpaca key (aliases: ALPACA_KEY_ID, ALPACA_API_KEY_ID, ALPACA_API_KEY).
APCA_API_SECRET_KEY	Alpaca secret (aliases: ALPACA_SECRET_KEY, ALPACA_API_SECRET_KEY).
TRADE_BASE	Must be https://api.alpaca.markets. Paper endpoints rejected.
DATA_BASE	https://data.alpaca.markets
API_TOKEN	Required in production. Protects every mutating endpoint and most debug endpoints.

8.2 Strategy economics

Variable	Default	What it does
TARGET_NET_PROFIT_BPS	8	Floor for the per-trade exit target after fees. Clamped to [5, 50].
SIGNAL_TARGET_FRACTION	1.0	Fraction of OLS projection captured. 30-day backtest A/B: 1.0 = +5.73 bps/entry v
SIGNAL_SIZED_EXIT_ENABLED	true	Per-trade TP from projection. OFF = fixed TARGET_NET_PROFIT_BPS for every
SIGNAL_TARGET_MAX_NET_BPS	50	Cap on the per-trade signal-sized net target.
FEE_BPS_ROUND_TRIP	40	~25 bps taker entry + ~15 bps maker exit.
PROFIT_BUFFER_BPS	5	Cushion in entry edge gate.
MIN_NET_EDGE_BPS	2	Minimum expected net edge before buying.
MIN_PROJECTED_BPS_TO_ENTER	15	Hard floor on OLS projection. ~3x slippage budget.
MIN_VOLUME_RATIO_TO_ENTER	1.0	Recent-window volume must >= lookback mean.
MAX_BTC_LEAD_LAG_DROP_BPS	-10	Refuse alts when BTC last-5-bar return <= threshold.
MIN_PORTFOLIO_UNREALIZED_PCT_TO_ENTER	-2.0	Pause all entries when book aggregate < -2.0% unrealized.
PORTFOLIO_SIZING_PCT	0.10	Fraction of equity per trade.
MIN_TRADE_NOTIONAL_USD	1	Dust floor.
MIN_SIZING_FRACTION_OF_TARGET	0.6	Skip when cash-clamped notional is < fraction of target.
BREAKEVEN_TIMEOUT_MS	14 400 000	Staircase decay window (4h).
STAIRCASE_EXIT_ENABLED	true	Linear TP -> break-even decay. OFF = one-shot break-even-replace at T.
STAIRCASE_REPOST_TOLERANCE_BPS	3	Min drop before cancel/repost.
STOP_LOSS_ENABLED	false	OFF = no realised losses by default.
STOP_LOSS_BPS	100	Cap on vol-scaled stop.
VOL_SCALED_STOP_ENABLED	true	Size stop from entry-time sigma.
STOP_LOSS_VOL_K	1.0	Number of sigma in the formula.
STOP_LOSS_HORIZON_BARS	60	sigma integration horizon (1m bars).
STOP_LOSS_BPS_FLOOR	20	Floor for vol-scaled stop.
ENTRY_SLIPPAGE_BPS	3	Slippage budget on entry.

Variable	Default	What it does
EXIT_SLIPPAGE_BPS	3	Slippage budget on exit.
CORRECTED_FILL_PROB_ENABLED	true	Use GBM barrier-hitting probability. OFF = legacy logistic_cdf proxy.
ENFORCE_GROSS_TARGET_FLOOR	true	Refuse trades that can't pay their own friction.
HONEST_EV_GATE_ENABLED	true	Charge non-fill branch a stuck-loss penalty.
STUCK_LOSS_ASSUMED_BPS	250	MTM loss assumed for non-recovering positions.
BARRIER_HORIZON_BARS	BREAKEVEN_TIMEOUT_MINS/60*60	Horizon for barrier-hitting probability.

8.3 Scanner and data

Variable	Default	What it does
ENTRY_SCAN_INTERVAL_MS	12 000	How often the entry loop runs.
EXIT_SCAN_INTERVAL_MS	15 000	How often exit/state poll runs.
ENTRY_QUOTE_MAX_AGE_MS	60 000	Reject quotes staler than this.
SPREAD_MAX_BPS	30	Skip symbols whose spread exceeds this.
PREDICT_BARS	20	Bars used in entry OLS.
VOLATILITY_MAX_BPS	100	Skip if realised vol exceeds this.
HTF_FILTER_ENABLED	true	Gate on higher-timeframe slope.
HTF_BARS	12	HTF lookback.
HTF_MIN_SLOPE_BPS_PER_BAR	1	HTF slope floor (bps/bar).
HTTP_TIMEOUT_MS	10 000	Per-request HTTP timeout.

8.4 Universe

Variable	What it does
ENTRY_UNIVERSE_MODE	Default dynamic in README posture - scanner walks every active Alpaca crypto pair (USD-
ENTRY_SYMBOLS_PRIMARY	Default 12 deep-liquidity pairs: BTC, ETH, SOL, AVAX, LINK, UNI, DOT, ADA, XRP, DOGE, LTC,
ALLOW_DYNAMIC_UNIVERSE_IN_PRODUCTION	Default true so production can opt into dynamic without an extra flag.

8.5 Toggles

Variable	Default	What it does
TRADING_ENABLED	true	Kill-switch for the buy path.
NET_EDGE_GATE_ENABLED	true	Disable to let all entries skip the edge gate.

Validated env-var list lives in `backend/config/validateEnv.js`. Non-secret production defaults live in `backend/config/liveDefaults.js`.

9. Operations

9.1 Setup

Requires Node 22 (`nvm use` in `backend/`).

```
cd backend
npm install # postinstall wires up .git-hooks
cp .env.example .env # fill in live Alpaca keys (never commit secrets)
npm test
npm run smoke
npm start
```

9.2 Tests and scripts

```
npm test # check:no-secrets + grouped suites
npm run smoke # local smoke test
npm run preflight # runtime-env check + smoke
npm run check:complexity # enforces line budget on trade.js
npm run reconcile # offline analysis: predicted vs realised
npm run backtest # replay strategy on real Alpaca bars
```

CI runs on every push/PR to `main`: backend `npm ci -> lint -> test -> runtime env sanity check`, frontend `npm ci` install-only smoke. See `.github/workflows/ci.yml`.

9.3 Production deployment (Render)

- Set every secret (`APCA_API_KEY_ID`, `APCA_API_SECRET_KEY`, `API_TOKEN`) directly in the Render env. Never in git - the pre-commit hook in `.git-hooks/pre-commit` blocks obvious cases.
- Run `npm run check:runtime-env` to validate config.
- Leave `ENTRY_UNIVERSE_MODE` unset (default *dynamic*) so the scanner walks every active Alpaca USD-quoted crypto pair minus stablecoins. Tier-1-tight spread/freshness gates filter the long tail.
- After deploy, `GET /debug/runtime-config` (token-protected) is the source of truth for what the live process actually sees. Verify `effectiveUniverseMode=dynamic` and `scanSymbolsCount` on the order of 30+ in the `startup_truth_summary` log line.

9.4 Docker

```
cd backend
docker build --build-arg GIT_COMMIT=$(git rev-parse --short HEAD) -t magic-backend .
docker run --rm -p 3000:3000 --env-file .env magic-backend
```

Render currently builds without the Dockerfile.

10. Known constraints and structural limitations

- Rate limiting (`backend/rateLimit.js`) is in-memory and per-process. Single-instance only.
- The Frontend is read-only diagnostic. It cannot place or modify orders.
- `backend/trade.js` is large; `npm run check:complexity` enforces a soft line cap.
- Crypto markets are 24/7 - there is no "market closed" safe window.
- No cross-symbol correlation guard. When the scanner walks 30+ pairs the engine can become long the same beta on multiple symbols simultaneously. The portfolio-drawdown gate is a coarse proxy that pauses *new* entries once correlated open positions have started bleeding - it doesn't prevent the first N entries from clustering.

10.1 Structural limitation of "small TP + long-hold tail"

Honest expectancy under realistic 1m crypto volatility (sigma ~ 12 bps/min) is **negative in flat or adverse drift regimes**, even though the engine *appears* loss-free because no realised loss is ever booked. Stuck positions accumulate negative MTM that the engine never crystallises. The cost-floor gate and corrected fill probability raise the bar entries must clear; they do not change the structural payoff. See section 7 for the simulator output.

11. Hard project rules

- **Keep README.md current.** Any PR that touches trading behaviour, default values for documented env vars, the "What the bot does NOT do" list, or top-level repo layout must update `README.md` in the same commit.
- **Never commit Alpaca credentials or `API_TOKEN` values.** Pre-commit hook blocks obvious cases; never bypass with `--no-verify`.
- **Live trading only.** `TRADE_BASE` must point at `https://api.alpaca.markets` in production. Paper endpoints are explicitly rejected. Don't add fallbacks that re-allow paper.
- **Don't re-introduce dead knobs as if they're real.** If you document a feature, the feature must actually be wired. The current backend has substantial doc-vs-code drift; do not make it worse.
- **Don't add stop-loss, max-hold, or force-exit logic without explicit user instruction.** The "walk away after placing the GTC sell" behaviour is intentional design, not a missing feature.

End of specification. Canonical reference: `README.md` at the repository root. Diagrams in this PDF are produced by `docs/generate_spec_images.py`; re-run that script and `docs/generate_spec_pdf.py` to regenerate.